D3.3
Integration of performance measuring tools

Project Title	dealii-X: an Exascale Framework for Digital Twins of the Human Body
Project Number	101172493
Funding Program	European High-Performance Computing Joint Undertaking
Project start date	1 October 2024
Duration	27 months

dealii-X has received funding from the European High-Performance Computing Joint Undertaking Programme under grant agreement Nº 101172493

Deliverable title	Integration of Performance Measuring Tools
Deliverable number	D3.3
Deliverable version	v1
Date of delivery	February 28, 2026
Actual date of delivery	February 28, 2026
Nature of deliverable	Demonstrator, pilot, prototype
Dissemination level	Public
Work Package	WP3
Partner responsible	BADW-LRZ

Abstract	This report details the software framework targeting performance measurement tools in the dealii-X project. The framework is made generic to support a range of European supercomputers on the exascale threshold.
Keywords	performance; analysis; energy; accelerators

Document Control Information

Version	Date	Author	Changes Made
0.1	Feb 10, 2026	Ivan Pribec	Initial document plan
0.2	Feb 27, 2026	Ivan Pribec	Benchmark & measurement tool description
0.3	Feb 28, 2026	M. Kronbichler	Minor adjustments
1.0	Feb 28, 2026	Ivan Pribec	Final version

Approval Details

Approved by: Martin Kronbichler
Approval Date: February 28, 2026

Distribution List

- Project Coordinators (PCs)
- Work Package Leaders (WPLs)
- Steering Committee (SC)
- European Commission (EC)

Disclaimer: This project has received funding from the European Union. The views and opinions expressed are those of the author(s) only and do not necessarily reflect those of the European Union or the European High-Performance Computing Joint Undertaking (the “granting authority”). Neither the European Union nor the granting authority can be held responsible for them.

Contents

1 Introduction

This deliverable provides a software framework demonstrator for the performance analysis of large-scale simulations on EuroHPC systems including accelerators. The intention is to provide a portable example (in terms of different hardware and software frameworks) which is applicable to a broad range or European supercomputing, thus supporting the transition to exascale computing.

The diversity of accelerators from multiple vendors (Nvidia, AMD, Intel) deployed at EuroHPC centers and also national HPC sites poses a significant challenge for scientists and engineers in terms of needed developments effort to obtain correct, robust and performant production codes for large-scale scientific computing.

1.1 Purpose of the Document

The purpose of the demonstrator is to provide a standardized blueprint for both library and application developers within WP1 and WP2 of the dealii-X project. The demonstrators are also of relevance to other stakeholders in the field of finite element software and GPU computing.

We utilize three distinct performance-measuring tools, listed here in descending order of complexity and implementation effort:

- Application-level profiling and roofline analysis
- Instrumentation-based tools
- Sampling-based tools

Understanding the trade-offs between these tools – specifically regarding data collection granularity, runtime overhead, and degree of automation – is essential for an informed selection of the appropriate tool for a certain performance analysis task. Integrating these performance analysis workflows early in the software development cycle is critical for ensuring codebases remain performant and portable across the diverse heterogenous environments of the EuroHPC ecosystem.

1.2 Structure of the Document

This document is organized into the following three sections:

- Section 2: [Poisson Solver Proxy App]
- Section 3: [Performance Analysis Tools]
- Section 4: [Energy Measurement]

2 Poisson Solver Proxy App

The demonstrator utilizies a Poisson solver proxy based on the CEED Bakeoff Problems (BP), similar to BP3. It solves the Poisson equation,

$$-\nabla^2 u = f \quad \text{in } \Omega,\, u = 0 \text{ on } \partial\Omega$$

over a 3-d box geometry, $\Omega = [0,1]^3$. The domain is discretized with hexahedral finite elements. The finite element space consists of tensor-product polynomials using nodal basis function at Gauss–Legendre–Lobatto (GLL) knots. For a polynomial degree p, there are $p+1$ points per spatial dimension on each element.

To ensure the proxy app is representative of the complex geometries found in biological models, the implementation assumes deformed elements. This means the solver must handle the full metric terms (Jacobian) of the transformation from physical to iso-parametric coordinates.

Given the self-adjoint nature of the Laplace operator, we employ a Conjugate Gradient (CG) solver. The system matrix is never assembled, instead the bilinear forms are evaluated in a matrix-free fashion which is more efficient on modern CPU and GPU architectures. Following the original CEED Bakeoff Problem setup, non-preconditioned CG is used, as the main purpose of the benchmark is for software optimization and performance analysis purposes.

For actual models, which may require computational meshes with billions of degrees of freedom, the computational speed is perfomed. We envision the use of efficient multigrid-based preconditioners, which are essential for “propagating information” quickly across the mesh thereby reduing the total number of CG iterations needed before convergence. Recent enhancements of the portable GPU multigrid preconditioners are described in the report for deliverable D1.7.

It is worth pointing out that the setup shares architectural similarities with the High-Performance Conjugate Gradient (HPCG) benchmark, although they differ in the choice of discretization method and intent. The HPCG benchmark utilizes a 27-point stencil finite difference method, while the BP3 proxy uses (high-order) finite elements. HPCG is typically limited by memory bandwidth due to the explicit (sparse) matrix stroage. In contrast, the matrix-free approach in the BP puts emphasis on arithmetic intensity and floating-point throughput. Unlike HPCG, which

employs a symmetric Gauss-Seidel preconditioner, the BP benchmark uses non-preconditoner CG.

2.1 BP Results

The Bakeoff Problem demonstrator app can be obtained using the commands in the listing below:

Listing 1: Script for building the BP3.5 demo

```
git clone https://github.com/dealii-X/benchmarks.git
cd benchmarks/bakeoff_problems_dealii

# check out specific commit (git checkout <commit_hash>)

cmake -B build -S src \
    -DCMAKE_CXX_COMPILER="$(which mpicxx)" \
    -DDEAL_II_DIR=<dealii_installation_path>
cmake --build build
```

We use application-level profiling using timers to measure the throughput of matrix-vector products and iterative solver steps.

The main quantities of interest are the run time as a function of the problem size and the behavior across different problem and machine sizes.

We have reported extensive results on a variety of Nvidia, AMD and Intel hardware in the report for deliverable D1.7. Here, we show one complementary result, based on outcome from a moderate degree of hardware tuning based on the Kokkos kernels. Fig. 1 shows the application throughput of the BP3.5 benchmark on the Intel Data Center GPU Max 1550, replicating a similar case shown in the deliverable report D1.7 for the Nvidia GH200. It can be seen that the throughput per GPU is considerably lower than the 8–10 GDOF/s on the Hopper GPU, but the data is nonetheless encouraging in the sense that portability with decent performance could be achieved. Further steps are work in progress.

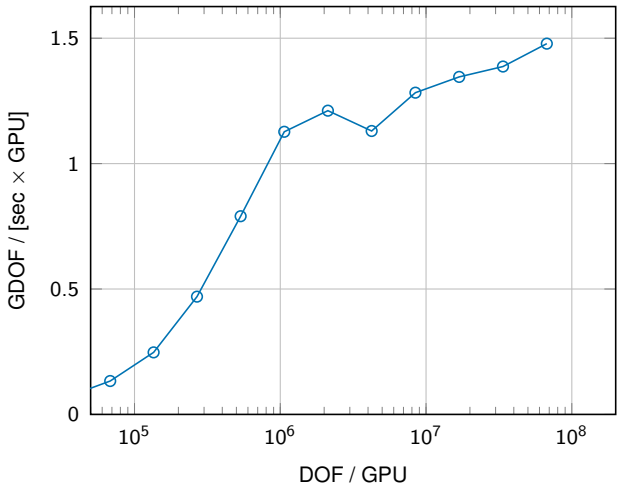

Figure 1: Throughput of benchmark BP3.5 with polynomial degree $p = 4$ (Gauss quadrature, $p+1$ quadrature points per direction) on 8 MPI ranks of Intel Data Center GPU Max 1550 (“Ponte Vecchio”).

3 Performance Analysis Tools

The complexity of both the software in question and the target hardware limit what can be inferred solely using application-based performance reporting.

Luckily, a growing amount of both vendor-provided and open-source software can provide in-depth analysis capabalities, based on hardware performance counters, micro-benchmarking, and automatic analysis capabilities which can make performance optimization easier.

In the following sections, we provide a description and usage examples of tools that we have used for the optimization of the Bakeoff Problems and the underlying Bakeoff Kernels.

The tools we have used so far are

- NVIDIA Nsight Systems,
- Intel VTune Profiler, and
- LIKWID Performance Tools.

3.1 NVIDIA NSight Systems

NSight™ Systems provides a system-wide timeline view, showing how the CPU and GPU execute the program. It also provides statistics about API calls, overhead and memory transfers (host-to-device/device-to-host).

To create a profile, we launch the application with the command[1]

```
> nsys profile <target> [target-options]
```

Further command switch options can be provided to control what is collected.

It is useful to collect profiling tool invocations either in the project readme file or scripts for future reference, a habit

3.2 Intel® VTune™ Profiler

The Intel® VTune™ Profiler[2] is a performance analysis tool providing profiling and analysis capabilities to understand application performance on Intel CPUs and GPUs. The profiler can be used both as a graphical or command-line application, offering a visual timeline and microarchitectural analysis to identify performance bottlenecks.

3.2.1 Intel Application Performance Snapshot

The Intel Application Performance Snapshot (APS) is a subtool from the VTune profiler (Linux OS only) that can provide a quick overview of compute-intensive applications running on Intel hardware. It can be used to check if a given application is CPU, Memory, or I/O bound before moving to a more fine-grained analysis.

To launch data collection with APS the command is

```
> aps --result-dir=<foldername> <target> [target-options]
```

[1] We use <> to denote mandatory arguments, and [] to denote optional arguments

[2] https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler.html

After finishing, a report is printed to the output terminal. In addition an HTML report is generated, which can be viewed in a supported browser (see Figure 2).

By default, APS collects statistics of the whole application run. To enable collection only for a specific region (such as a solver loop), the source code can be instrumented with the Intel Tracing Technology (ITT) API. This is particularly useful in large scale finite-element applications to ignore the preprocessing phase (mesh setup). As shown in Listing 2, we can start APS in paused mode and resume performance counter collection only for the region of interest.

Listing 2: Minimal instrumentation example using the Intel-specific ITT API

```
#include <ittnotify.h>

int main() {
    // Preprocessing, setup
    setup_mesh();

    __itt_resume(); // Start data collection
    run_solver();   // Main solver iteration loop
    __itt_pause();  // Stop data collection

    // Post-processing
    write_result();
    return 0;
}
```

An example of the APS output is shown in Fig. 2. Due to the coarse, application-level profiling granularity, the performance snapshot must be interpreted with care. For instance, in a GPU-accelerated benchmark, low physical utilization is expected. Another source of interpretation errors is that APS scales its metrics against the full hardware resources of the system (e.g., in SuperMUC-NG Phase 2, all four GPUs of a node). In tests targeting only a single GPU device, this results in “artificially” low utilization.

3.2.2 VTune Profiler Hotspots and GPU Analysis

For codes utilizing Intel GPUs, VTune provides specialized collectors to analyze execution on the offload device.

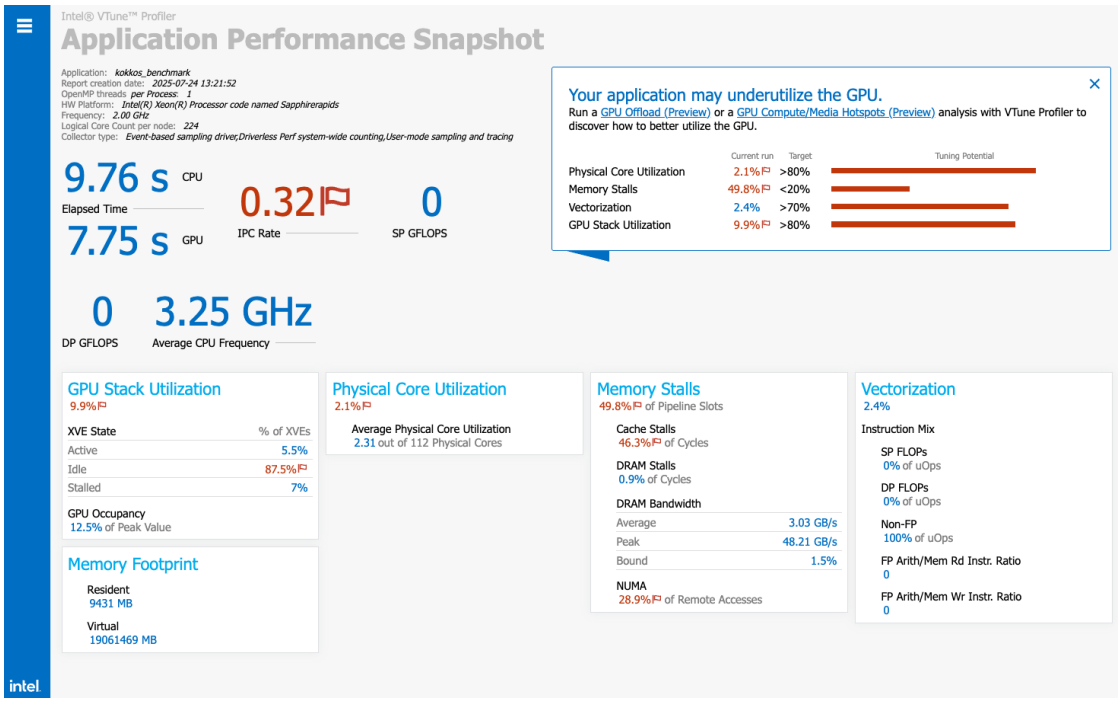

Figure 2: Example output from an early iteration of the Bakeoff Kernels using the Kokkos framework on SuperMUC-NG Phase 2 (Intel “Ponte Vecchio” GPUs).

```
> vtune -quiet -collect gpu-offload -r vtune_gpu_offload <executable>
> vtune -quiet -collect gpu-hotspots -r vtune_gpu_hotspots ...
```

The results are typically inspected via the VTune GUI, providing a powerful timeline view.

3.3 Intel® Advisor

The tool also support a command-line interface (CLI), useful to collect results on remote systems and/or under batch-scheduling constraints.

Collection of the roofline data is performed using the command

```
> advisor --collect=roofline --profile-gpu -- <executable>
```

To obtain line profiling data linked to the source file, the executable must be (re)compiled with the following `icpx` options: `-fdebug-info-for-profiling -gline-tables-only`.

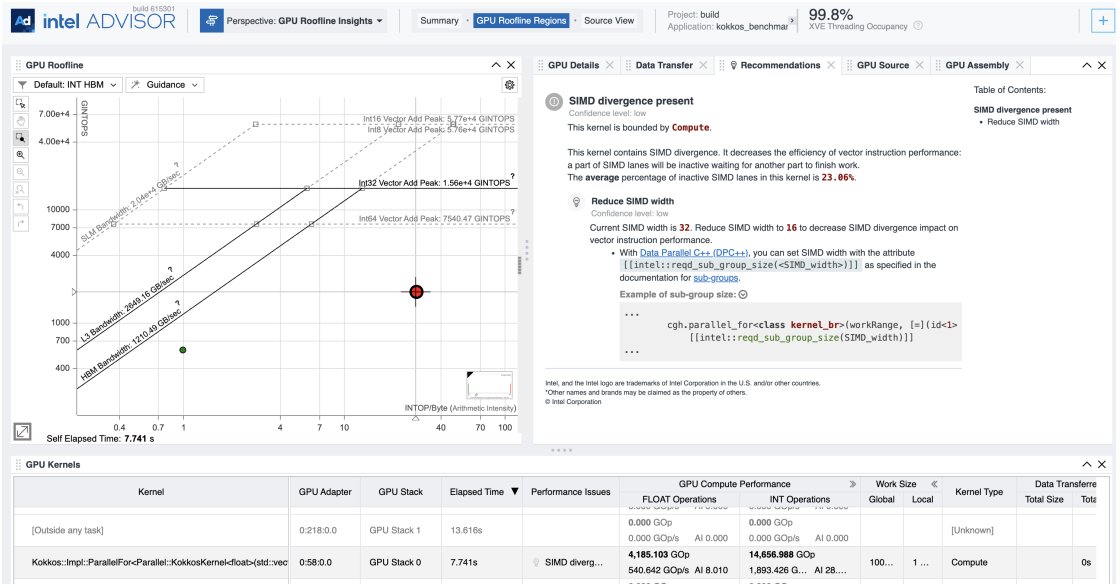

Figure 3: Exemplary view of Bakeoff Kernel result in Intel Advisor.

3.4 LIKWID Performance Tools

The LIKWID Performance Tools[3] developed at the Erlangen National High Performance Computing Center (NHR@FAU) are a suite of command-line applications for performance monitoring and benchmarking. They provide a lightweight abstraction for analyzing hardware performance counters across major CPU and GPU architectures. Within our demonstrator kernels, LIKWID is used to measure the arithmetic intensity and througput of high-order finite element kernels.

We make use of the LIKWID Marker API which enables the measuring of named regions of code. This allows us to isolate specific computational bottlenecks, such as the evaluation of the bilinear finite element forms, right-hand side evaluation, and the iterative linear equation solver. As shown in Listing 3, the instrumention is only minimally intrusive. The regions of interest are marked for collection by calling the Marker API functions or function-like macros, which allow the performance monitoring to be enabled or disabled using the preprocessor (`-D LIKWID_PERFMON`.).

[3] https://hpc.fau.de/research/tools/likwid/

Listing 3: Minimal example of using LIKWID Marker API for CPU instrumentation

```
#ifdef LIKWID_PERFMON
#include <likwid-marker.h>
#else
  // Empty definitions go here
#endif

int main() {
  LIKWID_MARKER_INIT;
  LIKWID_MARKER_THREADINIT;
  LIKWID_MARKER_START("matvec");
  /* region of interest we measure */
  LIKWID_MARKER_STOP("matvec");
  LIKWID_MARKER_CLOSE;
  return 0;
}
```

After the code has been instrumented and built accordingly, the measurements are collected by invoking the executable with the LIKWID performance counter tool:

```
> likwid-perfctr -C0-71 -g <group> -m <executable>
```

The output groups can be selected using the command-line options, and normally include metrics including floating point operations per second and memory-bandwidth related metrics (see Listing 4). We have also experimented with running LIKWID with MPI, using the command

```
likwid-mpirun -n 72 -g MEM_DP -m <executable>
```

for the example of a 72-core CPU system.

The instrumented executable is then used within a “measure-refine” cycle, analogous to the standard “edit-compile-run” cycle of software development, testing the impact of different optimizations (e.g. vectorization, cache-blocking) and immediately re-measuring to quantify the speedup and also inform the (numerical) algorithm design. Using the statistics collected by `likwid-perfctr` and related LIKWID tools, the scientific programmer can also create empirical roofline plots.

Listing 4: Output of `likwid-mpirun` for a BP3-inspired benchmark on an Intel Xeon Platinum 8360Y CPU (2 sockets, 72 cores used).

```
+----------------------------------------+-------------+------------+
|                 Metric                 |     Sum     |    Min     |
+----------------------------------------+-------------+------------+
|        Runtime (RDTSC) [s] STAT        |    417.3958 |     5.7865 |
|        Runtime unhalted [s] STAT       |    419.5135 |     5.7372 |
|            Clock [MHz] STAT            |  174094.7702 |  2386.7181 |
|                CPI STAT                |     24.6792 |     0.3352 |
|             Energy [J] STAT            |   2889.3940 |          0 |
|             Power [W] STAT             |    498.9132 |          0 |
|          Energy DRAM [J] STAT          |    160.2279 |          0 |
|           Power DRAM [W] STAT          |     27.6661 |          0 |
|            DP [MFLOP/s] STAT           | 1.797120e+06 | 24937.2927 |
|          AVX DP [MFLOP/s] STAT         | 1.779952e+06 | 24699.4630 |
|          Packed [MUOPS/s] STAT         |  222503.4982 |  3087.5617 |
|          Scalar [MUOPS/s] STAT         |   17163.6339 |   230.6502 |
|  Memory read bandwidth [MBytes/s] STAT |   92605.7665 |          0 |
|  Memory read data volume [GBytes] STAT |    536.3158 |          0 |
| Memory write bandwidth [MBytes/s] STAT |   9989.8876 |          0 |
| Memory write data volume [GBytes] STAT |     57.8554 |          0 |
|    Memory bandwidth [MBytes/s] STAT    |  102595.6540 |          0 |
|    Memory data volume [GBytes] STAT    |    594.1713 |          0 |
| Operational intensity [FLOP/Byte] STAT |      0.9741 |          0 |
|      Vectorization ratio [%] STAT      |   6684.3808 |    92.7566 |
+----------------------------------------+-------------+------------+
```

4 Energy Measurement

Energy consumption is an increasingly critical metric in modern computing. While often secondary for consumer-grade desktop software, the immense scale of High-Performance Computing (HPC) systems means that electricity usage accounts for a significant portion of total operational costs.

While hardware advancements have steadily improved performance-per-watt over recent decades, software efficiency remains a dominant factor. A prominent example is GPU memory optimization: utilizing coalesced loading patterns not only accelerates execution but also drastically reduces the energy footprint compared to unoptimized, strided access. However, the interplay of clock frequencies, computational intensity, and parallelism makes obtaining reproducible energy measurements a complex task for end-users.

4.1 p3em

p3em[4] (standing for Permitted, Portable, Parsable Energy Meter) offers a simple API for measuring energy across heterogeneous devices Cielo et al. [2025]. It is designed as a minimalistic, header-only C++ library, facilitating easy integration into existing codebases. Callers of the API create instances of the `p3em` class, which serves as an "energy-meter".

p3em operates by spawning a background thread that manages a power-measurement loop. This thread initiaties a sub-process which interfaces with vendor-specific APIs or OS-specific energy measurement tools (such as the NVIDIA tool `nvidia-smi` or Linux `perf`) to periodically query the device energy usage.

To maintain a hardware-agnostic interface, p3em requires the sub-process to output a single monotonically increasing value representing the total energy consumption in Joules. This value could be calculated via numerical integration of instantaneous power samples[5] or alternatively, the sub-process may directly report hardware-level energy counters if supported by the device. The background thread captures these value from the sub-process output, providing a synchronized running total for the given region of interest. The `p3em.sh` script provided in the library's

[4] https://github.com/svtcli/p3em
[5] In practice, these values may already be averaged by the device driver

repository serves as a versatile template, containing pre-configured commands for several platforms, including Nvidia, Intel, and AMD (GPU) architectures, which can be toggled depending on the available hardware.

The listing 5 demonstrates how to instrument a C++ ”region of interest” (ROI) using p3em. Conceptually, usage of the API is similar to using a RAII-based Timer class. Further usage examples in C, Fortran and Python can be found at the p3em source repository.

Listing 5: Energy measurement instrumentation using p3em

```
#include <cstdlib>
#include <iostream>
#include <p3em.hpp>

int main() {

    // Determine sub-process script location via envvar or
       default path
    auto p3em_script_path = []() -> std::string {
        const char* path = std::getenv("P3EM_SCRIPT");
        return path ? path : "p3em.sh";
    }();

    // Begin Region of Interest (ROI)
    {
        // Constructor initializes the background thread
        auto em = p3em(p3em_script_path);
        for(int i = 0; i < niters; ++i) {
            // ... Perform heavy computational kernels ...
        }
        // Retrieve and report energy usage
        std::cout << "Latest value:" << em.getLatestValue() <<
           '\n';

        // Meter stops when 'em' goes out of scope (RAII)
    }

    return 0;
}
```

An example of a specialized user-provided script, measuring energy by calling the NVIDIA Management Library via the Python bindings (`pyNVML`) is in Listing 6.

Listing 6: Python measurement script using pyNVML to stream cumulative GPU energy consumption in Joules.

```
#!/usr/bin/env python3
from pynvml import *
import sys
import time

try:
    nvmlInit()
    handle = nvmlDeviceGetHandleByIndex(0)
    while True:
        # returns total energy in millijoules
        nrg = nvmlDeviceGetTotalEnergyConsumption(handle)
        # Output Joules to stdout
        print(nrg / 1000.0)
        sys.stdout.flush()
        time.sleep(0.1)
finally:
    nvmlShutdown()
```

4.2 Other tools

Beyond custom instrumentation with p3em, several established tools provide energy telemetry at different system levels.

The Slurm job scheduler can be configured to report energy consumption for a completed job. In practice, this will typically be a single integrated value for the entire job duration, thus lacking the granularity required to correlate the energy consumption with specific application phases or kernels. A second disadvantage is that this information may also aggregate energy usage across different hardware components in a node.

The `likwid-powermeter` tool, part of the LIKWID toolsuite, can be used to measure power usage over a specified duration (so-called stethoscope mode) or invoked as a wrapper to measure energy usage of a given process. The use of the tool is

primarily ceneted on Intel/AMD CPUs where the RAPL (Running Average Power Limit) interfaces is available to provide energy counters.

For GPU devices, power draw can be queried using the vendor-specific “system management interfaces” (SMI), such as `nvidia-smi`, `rocm-smi` (AMD), and `xpu-smi` (Intel). While these tools are indispensible for monitoring, they require a level of external orchestration and post-processing (as seen with the p3em subprocess concept) to provide the energy data desirable for fine-grained analysis.

5 Conclusion

This report has documented the workflow established in dealii-X for performance measurements and tuning. We have made considerable progress in reaching hardware limits in a range of benchmarks, and will further build on this basis for the upcoming developments.

References

Salvatore Cielo, Alexander Pöppl, and Ivan Pribec. Sycl for energy-efficient numerical astrophysics: the case of dpecho. *arXiv preprint arXiv:2508.14117*, 2025.